

COMSATS UNIVERSITY ISLAMABAD SAHIWAL CAMPUS

Department Computer Science

ASSIGNMENT 4

5/17/2026

SUBMITTED BY:

Zarmeen Rasool

SUBMITTED TO:

Sir.InaamulHaq

REGISTRATION NUMBER:

FA23-BCS-112

SECTION:

6th (B)

Topic:

Transactions using MongoDB

DATE OF SUBMISSION:

May 18, 2026

Table of Contents

1. Database And Collection Creation:	5
1.1 Insert sample account records:	6
1.2 Verify Records Enter Successfully:	7
1.3 Python Code to create databse and collection:	8
1.3.1 Output:	9
2. MongoDB queries and Python code to perform basic CRUD operations:	10
2.1 Insert Query in MongoDB:	10
2.2 Read Query in MongoDB:	10
2.3 Update Query in MongoDB:	12
2.3.1 Before Updation:	12
2.3.2 Implementation Query:	12
2.3.3 After Updation:	12
2.4 Delete Query in MongoDB:	13
2.4.1 Before Deletion Records:	13
2.4.2 Implementation of Deletion:	13
2.4.3 After Deletion:	14
2.5 Python Crud Operations:	14
2.5.1 Output:	15
3. Concurrent updates on same account without using concurrency control:	15

3.1	MongoDb Shell Simulation:	16
3.1.1	User A Reads Data:	16
3.1.2	User A Update:	16
3.1.3	User B Update:	17
3.1.4	Final Record:	18
3.2	Python Concurrency Program:	19
3.2.1	Output:	20
4.	Implement Optimistic Concurrency Control (OCC) in MongoDB:	20
4.1	MongoDb Shell Occ	21
4.1.1	Reset Record	21
4.1.2	OCC Update:	22
4.1.3	Conflict Update:	23
4.1.4	Conflict Result:	23
4.2	Python Occ Program:	24
4.2.1	Output:	25
5.	Transaction-based MongoDB and explain how ACID properties are maintained:	25
5.1	Acid Properties:	25
5.2	MongoDb Shell Transaction:	26
5.2.1	Replica Set:	26
5.2.2	Start Session:	28

5.2.3	Select Database:	28
5.2.4	Start Transaction:	28
5.2.5	Deduct Money:.....	29
5.2.6	Add Money:	30
5.2.7	Commit Transaction:	30
5.2.8	Verify Records:	30
5.2.9	Session End:.....	30
5.3	Python Transaction Program:	31
6.	Transaction rollback in case of insufficient balance or system failure:.....	32
6.1	MongoDb Shell Rollback:	32
6.1.1	Start Transaction:	32
6.1.2	Session DB:.....	32
6.1.3	Start Transaction:	32
6.1.4	Read Sender:	33
6.1.5	Check Balance:	33
6.1.6	Verify No Changes Occurred:	34
6.2	Python Rollback:.....	35
7.	MongoDB handles consistency, isolation, and concurrent access in distributed systems:	36
7.1	Consistency	36

7.2	Isolation.....	36
7.3	Concurrent Access	36
7.4	Distributed System Support	36
8.	Compare normal updates with transaction-based updates:	37
8.1	Advantages Of Normal Update:.....	37
8.2	Limitations:	38
8.3	Advantages Of Transactions:.....	38
8.4	S	38
9.	Role Of Versioning, Atomic Operations, And Transactions:	38
9.1	Versioning.....	38
9.2	Atomic Operations	38
9.3	Transactions:	39
9.4	Observations:	40
9.5	Challenges Faced:	40
10.	CONCLUSION.....	40

ASSIGNMENT 4

Question:

- Create a MongoDB database named BankDB and a collection named accounts. Insert sample account records containing the account holder name, balance, and version number.
- Write MongoDB queries and Python code to perform basic CRUD operations on the accounts collection.
- Simulate concurrent updates on the same account without using concurrency control and explain the resulting race condition or lost update problem.
- Implement Optimistic Concurrency Control (OCC) in MongoDB using a version field and demonstrate how conflicting updates are detected.
- Develop a transaction-based money transfer system between two accounts using MongoDB transactions and explain how ACID properties are maintained.
- Demonstrate transaction rollback in case of insufficient balance or system failure.
- Analyze how MongoDB handles consistency, isolation, and concurrent access in distributed systems.
- Compare normal updates with transaction-based updates in MongoDB and discuss their advantages and limitations.
- Explain the role of versioning, atomic operations, and transactions in concurrency control. Provide source code, execution screenshots, output results, and a brief report discussing implementation steps, observations, challenges, and conclusions.

Solution:

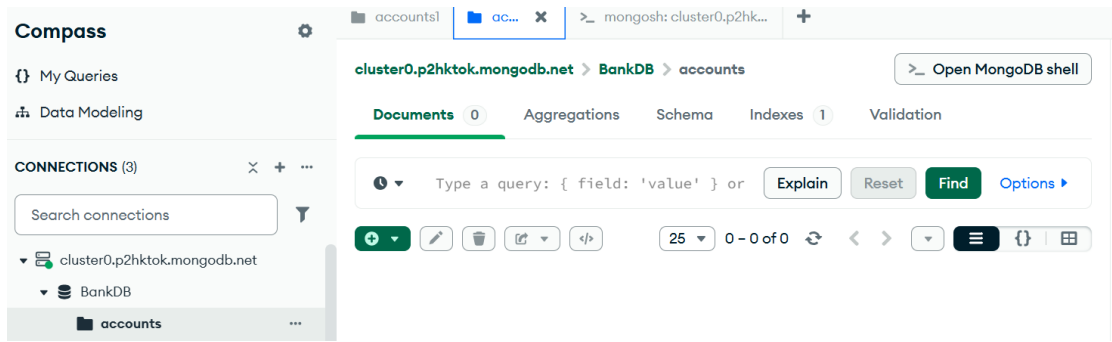
1. Database And Collection Creation:

A database is a collection of related data.

MongoDB is a NoSQL document database that stores data in BSON documents.

In this assignment:

- Database Name = BankDB
- Collection Name = accounts



1.1 Insert sample account records:

```
_MONGOSH
> use BankDB
< switched to db BankDB
> db["accounts"].find()
<
> db.accounts.insertMany([
  {
    account_no:1001,
    holder_name:"Ali Khan",
    balance:50000,
    version:1
  },
  {
    account_no:1002,
    holder_name:"Sara Ahmed",
    balance:30000,
    version:1
  },
  {
    account_no:1003,
    holder_name:"Usman Tariq",
    balance:45000,
    version:1
  }
])
```

```
{
  account_no:1003,
  holder_name:"Usman Tariq",
  balance:45000,
  version:1
}
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a09ed9185a3d9ff5fec6a3f'),
    '1': ObjectId('6a09ed9185a3d9ff5fec6a40'),
    '2': ObjectId('6a09ed9185a3d9ff5fec6a41')
  }
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

1.2 Verify Records Enter Successfully:

```
> db.accounts.find().pretty()
< {
  _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
  account_no: 1001,
  holder_name: 'Ali Khan',
  balance: 50000,
  version: 1
}
{
  _id: ObjectId('6a09ed9185a3d9ff5fec6a40'),
  account_no: 1002,
  holder_name: 'Sara Ahmed',
  balance: 30000,
  version: 1
}
{
  _id: ObjectId('6a09ed9185a3d9ff5fec6a41'),
  account_no: 1003,
  holder_name: 'Usman Tariq',
  balance: 45000,
  version: 1
}
```



```
}  
{  
  _id: ObjectId('6a09ed9185a3d9ff5fec6a41'),  
  account_no: 1003,  
  holder_name: 'Usman Tariq',  
  balance: 45000,  
  version: 1  
}  
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

1.3 Python Code to create database and collection:

```
bankdb.py X  
bankdb.py > ...  
1  from pymongo import MongoClient  
2  
3  client = MongoClient("mongodb://localhost:27017/")  
4  
5  db = client["BankDB"]  
6  
7  accounts = db["accounts"]  
8  
9  data = [  
10     {  
11         "account_no":1001,  
12         "holder_name":"Ali Khan",  
13         "balance":50000,  
14         "version":1  
15     },  
16     {  
17         "account_no":1002,  
18         "holder_name":"Sara Ahmed",  
19         "balance":30000,  
20         "version":1  
21     },  
22     {  
23         "account_no":1003,  
24         "holder_name":"Usman Tariq",
```

```
[
    {
        "account_no":1002,
        "holder_name":"Sara Ahmed",
        "balance":30000,
        "version":1
    },
    {
        "account_no":1003,
        "holder_name":"Usman Tariq",
        "balance":45000,
        "version":1
    }
]

accounts.insert_many(data)

print("Database and Collection Created")
```

1.3.1 Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\6thSemester\AdvDataBase\Python> python -u "e:\6thSemester\AdvDataBase\Python\bankdb.py"
Database and Collection Created
PS E:\6thSemester\AdvDataBase\Python>
```

2. MongoDB queries and Python code to perform basic CRUD operations:

2.1 Insert Query in MongoDB:

```
> db.accounts.insertOne({
  account_no:1004,
  holder_name:"Hina Malik",
  balance:25000,
  version:1
})
< {
  acknowledged: true,
  insertedId: ObjectId('6a09f08785a3d9ff5fec6a42')
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB> |
```

2.2 Read Query in MongoDB:

```
> db.accounts.find()
< {
  _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
  account_no: 1001,
  holder_name: 'Ali Khan',
  balance: 50000,
  version: 1
}
{
  _id: ObjectId('6a09ed9185a3d9ff5fec6a40'),
  account_no: 1002,
  holder_name: 'Sara Ahmed',
  balance: 30000,
  version: 1
}
{
  _id: ObjectId('6a09ed9185a3d9ff5fec6a41'),
  account_no: 1003,
  holder_name: 'Usman Tariq',
  balance: 45000,
  version: 1
}
}
```

```
{
  _id: ObjectId('6a09ed9185a3d9ff5fec6a41'),
  account_no: 1003,
  holder_name: 'Usman Tariq',
  balance: 45000,
  version: 1
}
{
  _id: ObjectId('6a09f08785a3d9ff5fec6a42'),
  account_no: 1004,
  holder_name: 'Hina Malik',
  balance: 25000,
  version: 1
}
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

2.3 Update Query in MongoDB:

2.3.1 Before Updation:

```
{
  _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
  account_no: 1001,
  holder_name: 'Ali Khan',
  balance: 50000,
  version: 1
}
```

2.3.2 Implementation Query:

```
> db.accounts.updateOne(
  {account_no:1001},
  {$set:{balance:55000}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Atlas atlas-i5ocl-g-shard-0 [primary] BankDB> |

2.3.3 After Updation:

```
▶ _id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
  account_no : 1001
  holder_name : "Ali Khan"
  balance : 55000
  version : 1
```



2.4 Delete Query in MongoDB:

2.4.1 Before Deletion Records:



The screenshot displays two document records in a MongoDB collection. Each record is shown in a light gray box with a play button icon on the left and edit, share, and delete icons on the right. The first record has an account number of 1003 and a holder named 'Usman Tariq'. The second record has an account number of 1004 and a holder named 'Hina Malik'.

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a41')
account_no : 1003
holder_name : "Usman Tariq"
balance : 45000
version : 1
```

```
_id: ObjectId('6a09f08785a3d9ff5fec6a42')
account_no : 1004
holder_name : "Hina Malik"
balance : 25000
version : 1
```

2.4.2 Implementation of Deletion:

```
> db.accounts.deleteOne({account_no:1004})
< {
  acknowledged: true,
  deletedCount: 1
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB> |
```

2.4.3 After Deletion:

🕒 Type a query: { field: 'value' } or [Explain](#) [Reset](#) [Find](#) [Options ▶](#)

[+](#) [✎](#) [🗑](#) [🔗](#) [📄](#) 25 1-3 of 3 [↺](#) [↻](#) [⌵](#) [☰](#) [🔗](#) [📄](#)

```

account_no : 1001
holder_name : "Ali Khan"
balance : 55000
version : 1

```

```

_id: ObjectId('6a09ed9185a3d9ff5fec6a40')
account_no : 1002
holder_name : "Sara Ahmed"
balance : 30000
version : 1

```

▶ [✎](#) [🔗](#) [📄](#) [🗑](#)

```

_id: ObjectId('6a09ed9185a3d9ff5fec6a41')
account_no : 1003
holder_name : "Usman Tariq"
balance : 45000
version : 1

```

2.5 Python Crud Operations:

```

bankdb.py × crud.py ×
crud.py > ...
1  from pymongo import MongoClient
2
3  client = MongoClient("mongodb://localhost:27017/")
4
5  db = client["BankDB"]
6
7  accounts = db["accounts"]
8
9  # INSERT
10
11 accounts.insert_one({
12     "account_no":1005,
13     "holder_name":"Ayesha Noor",
14     "balance":40000,
15     "version":1
16 })
17
18 print("Record Inserted")
19
20 # READ
21
22 print("\nAll Accounts")
23
24 for acc in accounts.find():
25     print(acc)

```

```

bankdb.py  crud.py  X
crud.py > ...
20 # READ
21
22 print("\nAll Accounts")
23
24 for acc in accounts.find():
25     print(acc)
26
27 # UPDATE
28
29 accounts.update_one(
30     {"account_no":1005},
31     {"$set":{"balance":45000}}
32 )
33
34 print("\nBalance Updated")
35
36 # DELETE
37
38 accounts.delete_one({"account_no":1005})
39
40 print("\nRecord Deleted")

```

2.5.1 Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS E:\6thSemester\AdvDataBase\Python> python -u "e:\6thSemester\AdvDataBase\Python\crud.py"
Record Inserted

All Accounts
{'_id': ObjectId('6a09ef94aac5a0bf8f9e737a'), 'account_no': 1001, 'holder_name': 'Ali Khan', 'balance': 50000, 'version': 1}
{'_id': ObjectId('6a09ef94aac5a0bf8f9e737b'), 'account_no': 1002, 'holder_name': 'Sara Ahmed', 'balance': 30000, 'version': 1}
{'_id': ObjectId('6a09ef94aac5a0bf8f9e737c'), 'account_no': 1003, 'holder_name': 'Usman Tariq', 'balance': 45000, 'version': 1}
{'_id': ObjectId('6a09f3521044bdc8d154d26d'), 'account_no': 1005, 'holder_name': 'Ayesha Noon', 'balance': 40000, 'version': 1}

Balance Updated

Record Deleted
PS E:\6thSemester\AdvDataBase\Python>

```

3. Concurrent updates on same account without using concurrency control:

Concurrency means multiple users accessing same data simultaneously.

Problem:

- Two users update same account together
- One update overwrites another

Called:

- Race Condition
- Lost Update Problem

Example

- Initial Balance:50000
- User AWithdraws:10000
- New balance:40000
- User B Deposits:5000
- New balance:55000
- Expected Final Balance:45000

But due to concurrency issue:

- Final balance may become: 40000 OR 55000

3.1 MongoDb Shell Simulation:

3.1.1 User A Reads Data:

```
> db.accounts.findOne({account_no:1001})
< {
  _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
  account_no: 1001,
  holder_name: 'Ali Khan',
  balance: 55000,
  version: 1
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB> |
```

3.1.2 User A Update:

```
▶ _id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
  account_no : 1001
  holder_name : "Ali Khan"
  balance : 55000
  version : 1
```



```
> db.accounts.updateOne(
  {account_no:1001},
  {$set:{balance:40000}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

 `_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')`
`account_no : 1001`
`holder_name : "Ali Khan"`
`balance : 40000`
`version : 1`



3.1.3 User B Update:

```
> db.accounts.updateOne(
  {account_no:1001},
  {$set:{balance:55000}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Atlas atlas-i5oclg-shard-0 [primary] BankDB >

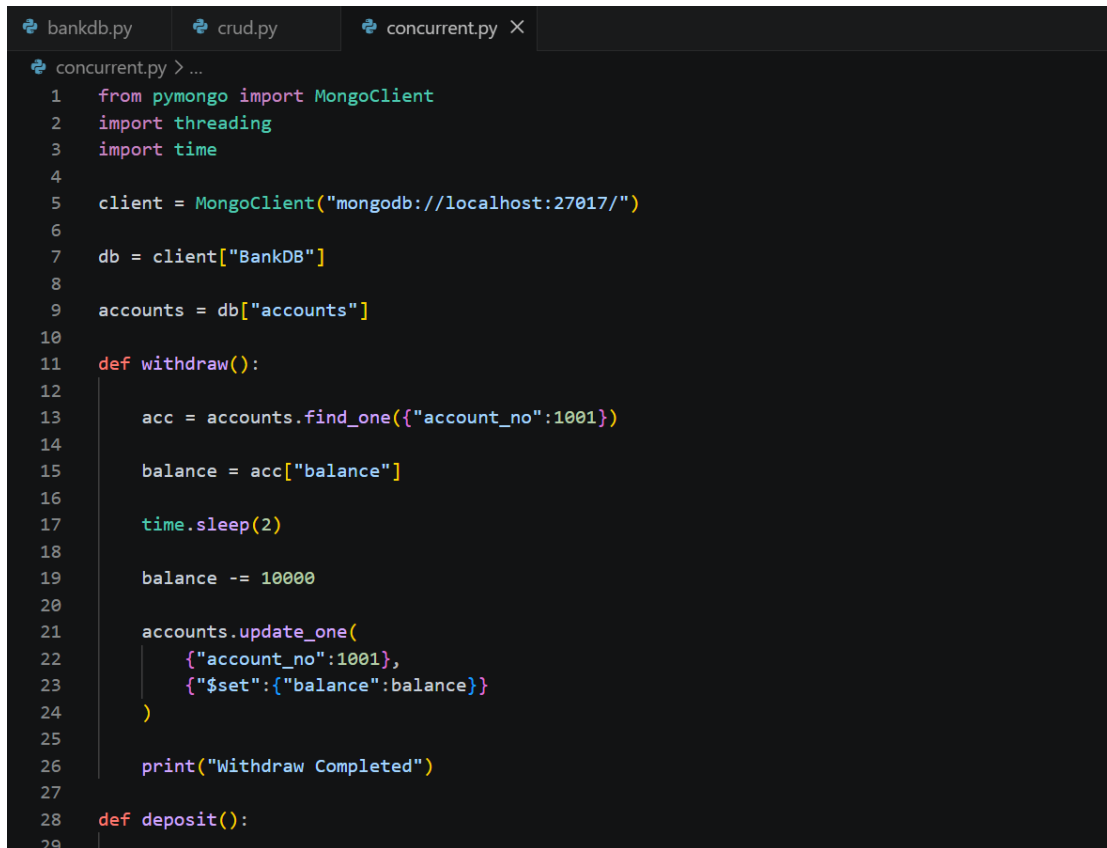
```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 55000
version : 1
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a40')
account_no : 1002
holder_name : "Sara Ahmed"
balance : 30000
version : 1
```

3.1.4 Final Record

```
> db.accounts.findOne({account_no:1001})
< {
  _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
  account_no: 1001,
  holder_name: 'Ali Khan',
  balance: 55000,
  version: 1
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

3.2 Python Concurrency Program:



The screenshot shows a code editor with three tabs: bankdb.py, crud.py, and concurrent.py. The concurrent.py tab is active, displaying a Python script for a concurrency program. The script imports pymongo and threading, connects to a MongoDB instance, and defines two functions: withdraw() and deposit(). The withdraw() function finds an account, sleeps for 2 seconds, and updates the balance. The deposit() function is partially visible at the bottom.

```
bankdb.py  crud.py  concurrent.py X
concurrent.py > ...
1  from pymongo import MongoClient
2  import threading
3  import time
4
5  client = MongoClient("mongodb://localhost:27017/")
6
7  db = client["BankDB"]
8
9  accounts = db["accounts"]
10
11 def withdraw():
12     acc = accounts.find_one({"account_no":1001})
13
14     balance = acc["balance"]
15
16     time.sleep(2)
17
18     balance -= 10000
19
20     accounts.update_one(
21         {"account_no":1001},
22         {"$set":{"balance":balance}}
23     )
24
25     print("Withdraw Completed")
26
27
28 def deposit():
29
```

```

concurrent.py > withdraw
11 def withdraw():
25
26     print("Withdraw Completed")
27
28 def deposit():
29
30     acc = accounts.find_one({"account_no":1001})
31
32     balance = acc["balance"]
33
34     time.sleep(1)
35
36     balance += 5000
37
38     accounts.update_one(
39         {"account_no":1001},
40         {"$set":{"balance":balance}}
41     )
42
43     print("Deposit Completed")
44
45 t1 = threading.Thread(target=withdraw)
46 t2 = threading.Thread(target=deposit)
47
48 t1.start()
49 t2.start()
50
51 t1.join()
52 t2.join()
53

```

```

28 def deposit():
41     )
42
43     print("Deposit Completed")
44
45 t1 = threading.Thread(target=withdraw)
46 t2 = threading.Thread(target=deposit)
47
48 t1.start()
49 t2.start()
50
51 t1.join()
52 t2.join()
53
54 final = accounts.find_one({"account_no":1001})
55
56 print("\nFinal Balance:", final["balance"])

```

3.2.1 Output:

```

python -u "E:\6thSemester\AdvDataBase\Python\concurrency_demo.py"
Deposit Completed
Withdraw Completed

Final Balance: 40000
PS E:\6thSemester\AdvDataBase\Python>

```

4. Implement Optimistic Concurrency Control (OCC) in MongoDB:

OCC uses a version field.

Steps:

- Read current version
- Update only if version matches
- Increment version after update

If another transaction already changed data:

- Update fails
- Conflict detected

4.1 MongoDB Shell Occ

4.1.1 Reset Record

```
> db.accounts.updateOne(  
  {  
    account_no:1001  
  },  
  {  
    $set:{  
      balance:50000,  
      version:1  
    }  
  }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 50000
version : 1
```

4.1.2 OCC Update:

```
> db.accounts.updateOne(
  {
    account_no:1001,
    version:1
  },
  {
    $set:{balance:45000},
    $inc:{version:1}
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 45000
version : 2
```

4.1.3 Conflict Update:

```
> db.accounts.updateOne(  
  {  
    account_no:1001,  
    version:1  
  },  
  {  
    $set:{balance:55000},  
    $inc:{version:1}  
  }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 0,  
  modifiedCount: 0,  
  upsertedCount: 0  
}  
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')  
account_no : 1001  
holder_name : "Ali Khan"  
balance : 45000  
version : 2
```

4.1.4 Conflict Result:

modifiedCount:0

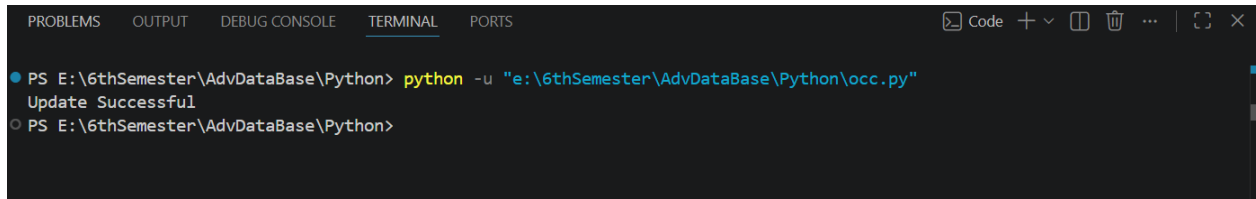
- Conflict detected
- Record already modified

4.2 Python Occ Program:

```
bankdb.py  crud.py  concurrency_demo.py  occ.py  X
occ.py > ...
1  from pymongo import MongoClient
2
3  client = MongoClient("mongodb://localhost:27017/")
4
5  db = client["BankDB"]
6
7  accounts = db["accounts"]
8
9  acc = accounts.find_one({"account_no":1001})
10
11 current_balance = acc["balance"]
12
13 current_version = acc["version"]
14
15 new_balance = current_balance - 5000
16
17 result = accounts.update_one(
18     {
19         "account_no":1001,
20         "version":current_version
21     },
22     {
23         "$set":{"balance":new_balance},
24         "$inc":{"version":1}
25     }
26 )
27
28 if result.modified_count == 1:
29     print("Update Successful")
```

```
occ.py > ...
14
15 new_balance = current_balance - 5000
16
17 result = accounts.update_one(
18     {
19         "account_no":1001,
20         "vDersion":current_version
21     },
22     {
23         "$set":{"balance":new_balance},
24         "$inc":{"version":1}
25     }
26 )
27
28 if result.modified_count == 1:
29     print("Update Successful")
30 else:
31     print("Conflict Detected")
```

4.2.1 Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS E:\6thSemester\AdvDataBase\Python> python -u "e:\6thSemester\AdvDataBase\Python\occ.py"
Update Successful
PS E:\6thSemester\AdvDataBase\Python>
```

5. Transaction-based MongoDB and explain how ACID properties are maintained:

A transaction is a group of operations executed as one unit.

MongoDB transactions maintain ACID properties.

5.1 Acid Properties:

Property	Meaning
Atomicity	All operations succeed or fail together
Consistency	Database remains valid
Isolation	Transactions do not interfere
Durability	Data remains permanent

5.2MongoDb Shell Transaction:

5.2.1 Replica Set:

```
> rs.status()
< {
  set: 'atlas-i5oclg-shard-0',
  date: 2026-05-17T17:45:13.499Z,
  myState: 1,
  term: Long('285'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
    lastCommittedWallTime: 2026-05-17T17:45:13.367Z,
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
    appliedOpTime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
    durableOpTime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
    writtenOpTime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
    lastAppliedWallTime: 2026-05-17T17:45:13.367Z,
    lastDurableWallTime: 2026-05-17T17:45:13.367Z,
```

```
},
members: [
  {
    _id: 0,
    name: 'ac-tfd1u4t-shard-00-00.p2hktok.mongodb.net:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 789807,
    optime: { ts: Timestamp({ t: 1779039912, i: 2 }), t: Long('285') },
    optimeDurable: { ts: Timestamp({ t: 1779039912, i: 2 }), t: Long('285') },
    optimeWritten: { ts: Timestamp({ t: 1779039912, i: 2 }), t: Long('285') },
    optimeDate: 2026-05-17T17:45:12.000Z,
    optimeDurableDate: 2026-05-17T17:45:12.000Z,
    optimeWrittenDate: 2026-05-17T17:45:12.000Z,
    lastAppliedWallTime: 2026-05-17T17:45:13.367Z,
    lastDurableWallTime: 2026-05-17T17:45:13.367Z,
    lastWrittenWallTime: 2026-05-17T17:45:13.367Z,
    lastHeartbeat: 2026-05-17T17:45:12.229Z,
    lastHeartbeatRecv: 2026-05-17T17:45:13.046Z,
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
```

```
    _id: 1,
    name: 'ac-tfd1u4t-shard-00-01.p2hktok.mongodb.net:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 789847,
    optime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
    optimeDate: 2026-05-17T17:45:13.000Z,
    optimeWritten: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
    optimeWrittenDate: 2026-05-17T17:45:13.000Z,
    lastAppliedWallTime: 2026-05-17T17:45:13.367Z,
    lastDurableWallTime: 2026-05-17T17:45:13.367Z,
    lastWrittenWallTime: 2026-05-17T17:45:13.367Z,
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1778250372, i: 1 }),
    electionDate: 2026-05-08T14:26:12.000Z,
    configVersion: 104,
    configTerm: 285,
    self: true,
    lastHeartbeatMessage: ''
```

```
],
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1779039913, i: 4 }),
    signature: {
      hash: Binary.createFromBase64('BpbM92o12aZW5DA1pHe02I5CRu4=', 0),
      keyId: Long('7610459603565805579')
    }
  },
  operationTime: Timestamp({ t: 1779039913, i: 4 })
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB >
```

5.2.2 Start Session:

```
> session = db.getMongo().startSession()
< {
  id: UUID('47c7cdfb-6ba8-4374-8dce-0415438ee0d5')
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB >
```

5.2.3 Select Database:

```
> sessionDB = session.getDatabase("BankDB")
< BankDB
Atlas atlas-i5oclg-shard-0 [primary] BankDB >
```

5.2.4 Start Transaction:

```
> session.startTransaction()
Atlas atlas-i5oclg-shard-0 [primary] BankDB >
```

5.2.5 Deduct Money:

🕒 ▼

Type a query: { field: 'value' } or

Explain

Reset

Find

Options ▶

⊕ ▼

✎

🗑

🔗 ▼

🔗

25 ▼ 1 - 3 of 3 ↺ < > ▼

☰ {} 🏠

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 45000
version : 2
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a40')
account_no : 1002
holder_name : "Sara Ahmed"
balance : 30000
version : 1
```

```
> session.startTransaction()
> sessionDB.accounts.updateOne(
  {account_no:1001},
  {$inc:{balance:-5000}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB> |
```

5.2.6 Add Money:

```
> sessionDB.accounts.updateOne(
  {account_no:1002},
  {$inc:{balance:5000}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

5.2.7 Commit Transaction:

```
}
> session.commitTransaction()
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

5.2.8 Verify Records:

 `_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')`
`account_no : 1001`
`holder_name : "Ali Khan"`
`balance : 40000`
`version : 2`

`_id: ObjectId('6a09ed9185a3d9ff5fec6a40')`
`account_no : 1002`
`holder_name : "Sara Ahmed"`
`balance : 35000`
`version : 1`

5.2.9 Session End:

```
> session.commitTransaction()
> session.endSession()
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

5.3 Python Transaction Program:

```
transaction.py > ...
1  from pymongo import MongoClient
2
3  client = MongoClient("mongodb://localhost:27017/")
4
5  db = client["BankDB"]
6
7  accounts = db["accounts"]
8
9  session = client.start_session()
10
11  try:
12
13      with session.start_transaction():
14
15          sender = accounts.find_one(
16              {"account_no":1001},
17              session=session
18          )
19
20          if sender["balance"] < 5000:
21              raise Exception("Insufficient Balance")
22
23          accounts.update_one(
24              {"account_no":1001},
25              {"$inc":{"balance":-5000}},
26              session=session
27          )
28
29          accounts.update_one(
```

```
20          if sender["balance"] < 5000:
21              raise Exception("Insufficient Balance")
22
23          accounts.update_one(
24              {"account_no":1001},
25              {"$inc":{"balance":-5000}},
26              session=session
27          )
28
29          accounts.update_one(
30              {"account_no":1002},
31              {"$inc":{"balance":5000}},
32              session=session
33          )
34
35          print("Transaction Successful")
36
37  except Exception as e:
38
39      print("Transaction Failed:", e)
40
41  finally:
42
43      session.end_session()
```


6. Transaction rollback in case of insufficient balance or system failure:

Rollback means undoing all operations if transaction fails.

Example:

- Insufficient balance
- System crash
- Network failure

No partial update occurs.

6.1 MongoDb Shell Rollback:

6.1.1 Start Transaction:

```
> session = db.getMongo().startSession()
< {
  id: UUID('47c7cdfb-6ba8-4374-8dce-0415438ee0d5')
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB >
```

6.1.2 Session DB:

```
}
> sessionDB = session.getDatabase("BankDB")
< BankDB
Atlas atlas-i5oclg-shard-0 [primary] BankDB >
```

6.1.3 Start Transaction:

```
> sessionDB = session.getDatabase("BankDB")
< BankDB
> session.startTransaction()
Atlas atlas-i5oclg-shard-0 [primary] BankDB >
```

6.1.4 Read Sender:

```
sender = sessionDB.accounts.findOne(  
  {account_no:1001}  
)  
{  
  _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),  
  account_no: 1001,  
  holder_name: 'Ali Khan',  
  balance: 40000,  
  version: 2  
}  
Atlas atlas-i5oclg-shard-0 [primary] BankDB> |
```

6.1.5 Check Balance:

```
> if(sender.balance < 100000)  
{  
  session.abortTransaction()  
  
  print("Transaction Aborted - Insufficient Balance")  
}  
< Transaction Aborted - Insufficient Balance  
Atlas atlas-i5oclg-shard-0 [primary] BankDB> |
```

6.1.6 Verify No Changes Occurred:

```
> db.accounts.find().pretty()
< {
  _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
  account_no: 1001,
  holder_name: 'Ali Khan',
  balance: 40000,
  version: 2
}
{
  _id: ObjectId('6a09ed9185a3d9ff5fec6a40'),
  account_no: 1002,
  holder_name: 'Sara Ahmed',
  balance: 35000,
  version: 1
}
{
  _id: ObjectId('6a09ed9185a3d9ff5fec6a41'),
  account_no: 1003,
  holder_name: 'Usman Tariq',
  balance: 45000,
  version: 1
}
```

6.2 Python Rollback:

```
transaction.py > ...
1  from pymongo import MongoClient
2
3  client = MongoClient("mongodb://localhost:27017/")
4
5  db = client["BankDB"]
6
7  accounts = db["accounts"]
8
9  session = client.start_session()
10
11  try:
12
13      with session.start_transaction():
14
15          sender = accounts.find_one(
16              {"account_no":1001},
17              session=session
18          )
19
20          if sender["balance"] < 10000:
21              raise Exception("Insufficient Balance")
22
23          accounts.update_one(
24              {"account_no":1001},
25              {"$inc":{"balance":-5000}},
26              session=session
27          )
28
29          accounts.update_one(
```

```
17          session=session
18      )
19
20      if sender["balance"] < 10000:
21          raise Exception("Insufficient Balance")
22
23      accounts.update_one(
24          {"account_no":1001},
25          {"$inc":{"balance":-5000}},
26          session=session
27      )
28
29      accounts.update_one(
30          {"account_no":1002},
31          {"$inc":{"balance":5000}},
32          session=session
33      )
34
35      print("Transaction Successful")
36
37  except Exception as e:
38
39      print("Transaction Failed:", e)
40
41  finally:
42
43      session.end_session()
```

7. MongoDB handles consistency, isolation, and concurrent access in distributed systems:

7.1 Consistency

MongoDB ensures:

- Correct data
- No invalid state
- Proper replica synchronization

7.2 Isolation

Transactions are isolated using:

- Snapshot isolation
- Document locking

Other users cannot see incomplete data.

7.3 Concurrent Access

MongoDB handles concurrency using:

- Atomic operations
- OCC
- Transactions
- Locking

7.4 Distributed System Support

MongoDB supports:

- Replica Sets
- Sharding

- Read Concern
- Write Concern

Benefits:

- High availability
- Scalability
- Fault tolerance

8. Compare normal updates with transaction-based updates:

Feature	Normal update	Transaction
Scope	Single document	Multiple documents
Rollback	No	Yes
Atomicity	Limited	Full
Consistency	Medium	High
Speed	Fast	Slower
Complexity	Easy	Complex

8.1 Advantages Of Normal Update:

- Fast
- Easy
- Low overhead

8.2 Limitations:

- No rollback
- Risk of inconsistency

8.3 Advantages Of Transactions:

- Strong consistency
- Reliable banking system
- Safe concurrent access

8.4

- Slightly slower
- More resource usage

9. Role Of Versioning, Atomic Operations, And Transactions:**9.1 Versioning**

Used in OCC.

Purpose:

- Detect concurrent modifications
- Prevent lost updates

9.2 Atomic Operations

MongoDB single document operations are atomic.

Example:

```
> db.accounts.updateOne(
  {account_no:1001},
  {$inc:{balance:1000}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Atlas atlas-i5oclg-shard-0 [primary] BankDB> |
```

```
▶ _id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
  account_no : 1001
  holder_name : "Ali Khan"
  balance : 41000
  version : 2
```



Safe concurrent update.

9.3 Transactions:

Provide:

- Multi-document consistency
- Rollback support
- ACID guarantees

Important in:

- Banking
- E-commerce
- Financial systems

9.4 Observations:

- Concurrent updates may overwrite data
- OCC successfully detects conflicts
- Transactions maintain consistency
- Rollback prevents invalid updates
- MongoDB provides reliable concurrency control

9.5 Challenges Faced:

- Configuring replica set
- Understanding transactions
- Managing concurrent operations
- Handling sessions in Python

10. CONCLUSION

MongoDB provides powerful concurrency control mechanisms including:

- Atomic operations
- Versioning
- Transactions

These features help maintain:

- Consistency
- Reliability
- Isolation
- Fault tolerance

MongoDB transactions are highly useful in enterprise and banking applications where data integrity is critical.